STUDENT PROJECT

STUDENT ASSIGNMENT TRACKER

Project Report and System Study

REACT • NODE.JS • EXPRESS • MONGODB

Prepared by	AKSHAY P SAJEEV
Register number	SHR25CS028
Course / Department	B.TECH / COMPUTER SCIENCE AND ENGINEERING
Institution	SAHRDAYA COLLEGE OF ENGINEERING AND TECHNOLOGY,KODAKARA
Project guide	DR.ANIL ANTONY
Academic year	2026-27

Hosted Frontend: https://akshay225725-bit.github.io/student-assignment-tracker/

Repository: https://github.com/akshay225725-bit/student-assignment-tracker

DECLARATION

I declare that this project report describes the Student Assignment Tracker application developed for academic purposes. Any external resources used are acknowledged in the references. The implementation scope stated in this report reflects the current application and should be checked against the final deployed version before submission.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members, and institution for their guidance and support. I also acknowledge the developers and maintainers of the open-source technologies used in this project, including React, Vite, Node.js, Express, Mongoose, and MongoDB.

ABSTRACT

Student Assignment Tracker is a full-stack web application designed to help students organize coursework and monitor assignment progress in one place. The interface provides a dashboard with assignment counts, a searchable and filterable assignment list, and forms to create and edit assignments. Each assignment can store a title, description, due date, priority, and status. A delete operation is also available from the interface.

The frontend is built with React and Vite and is published through GitHub Pages. The backend uses Node.js and Express to expose a REST-style API. Assignment records are stored in MongoDB through Mongoose, and the backend is hosted on Render. The browser communicates with the backend over HTTPS. During the deployment checks recorded for this project, the health route and assignments endpoint returned JSON, the hosted page loaded assignments, and creating an assignment followed by refreshing the page confirmed persistence. Edit, delete, search, and filter functions are part of the codebase and should be included in the final demonstration after being verified in the deployed environment.

KEYWORDS

Assignment management, student productivity, React, Vite, Express.js, Node.js, MongoDB, Mongoose, REST API, GitHub Pages, Render.

CONTENTS

	Title	Page
	Introduction	4
	Requirements and Feasibility	5
	System Design	6
	Implementation	8
	Testing and Validation	9
	Deployment and Operation	10
	Conclusion and Future Enhancements	11
	References	11

1. INTRODUCTION

1.1 Background

Students typically manage assignments across notebooks, reminders, course portals, and personal messages. When deadlines and progress are scattered across multiple places, it can be difficult to see what is pending or what needs attention. A small assignment tracker provides a single list and clear status indicators so a student can plan academic work more consistently.

1.2 Problem Statement

The problem addressed by this project is the lack of a simple, centralized view of assignment details and progress. The application needs to record basic assignment information, support status changes, and make existing records available after a browser refresh.

1.3 Objectives

- Provide a central dashboard for assignments.
- Create, view, update, and delete assignment records.
- Record due dates, priority, description, and progress status.
- Allow the user to search assignments and filter them by status.
- Display summary counts for all, pending, in-progress, and completed assignments.
- Store data in a backend database so records persist across refreshes.
- Deploy the frontend and backend so the application is accessible online.

1.4 Scope

The current scope is a single-user student workspace without a separate authentication workflow. It covers assignment tracking and basic progress management. Multi-user accounts, instructor dashboards, reminders, file uploads, grading, and calendar integrations are outside the implemented scope and are considered future enhancements.

1.5 Intended Users

The primary intended user is a student who wants a lightweight tool for tracking assignment deadlines and progress. The project is also intended as an educational example of connecting a React frontend, Express API, and document database.

2. REQUIREMENTS AND FEASIBILITY

2.1 Functional Requirements

	Requirement
01	Load and display assignments from the backend.
02	Create an assignment with title, description, due date, priority, and status.
03	Edit the details of an existing assignment.
04	Delete an assignment after user confirmation.
05	Search records by title or description.
06	Filter records by assignment status.
07	Show dashboard counts for assignment statuses.
08	Persist assignment data in MongoDB.
09	Provide a health endpoint to check backend availability.

2.2 Non-Functional Requirements

- Usability: clear labels, visible status and priority tags, and straightforward forms.
- Reliability: report API errors to the user rather than silently failing.
- Maintainability: separate frontend, backend routes, controllers, model, and database configuration.
- Portability: use Node.js and npm for local setup and managed hosting for deployment.
- Security: keep database credentials in environment variables and do not commit .env files.
- Performance: suitable for a small student project and moderate assignment volume.

2.3 Hardware and Software Requirements

Component	Requirement
Client device	Desktop/laptop or modern mobile device with a current browser
Runtime	Node.js and npm for local development
Frontend	React, Vite, JavaScript, CSS
Backend	Node.js, Express.js, cors middleware
Data access	Mongoose ODM
Database	MongoDB Atlas
Hosting	GitHub Pages for frontend; Render Web Service for backend
Version control	Git and GitHub

2.4 Feasibility

The project is technically feasible because it uses common JavaScript tooling and a REST API. It is operationally simple for students to use, and can be demonstrated using the available hosting tiers, subject to their limits. The codebase is compact enough to maintain and extend as an academic project.

3. SYSTEM DESIGN

3.1 Architecture Overview

The application is organized into three main layers. The React frontend runs in the browser and sends HTTP requests. The Express backend validates and processes requests through route handlers and controller functions. Mongoose communicates with MongoDB Atlas to retrieve or persist assignment documents.

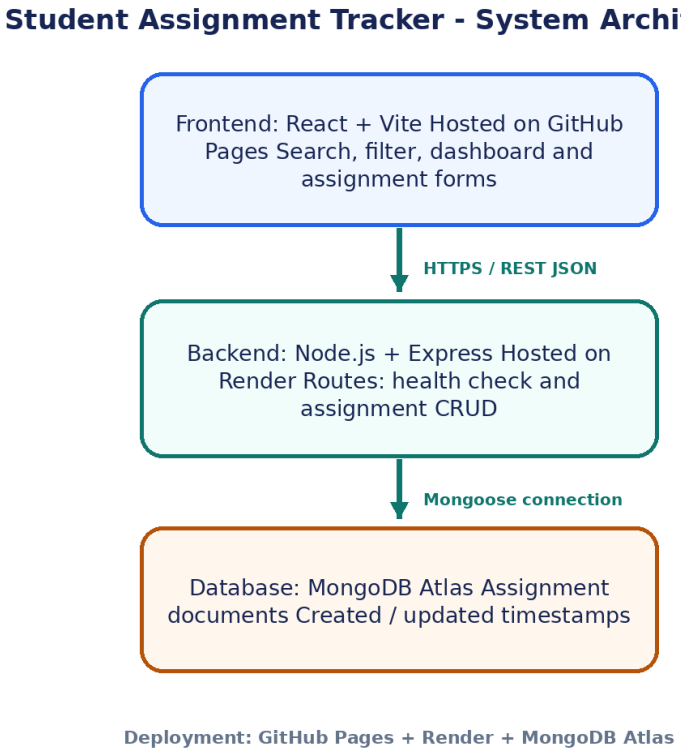


Figure 1. High-level deployment and request architecture.

3.2 Main Components

omponent	Responsibility
rontend	React components, application state, form handling, dashboard, search and status filters.
PI layer	Express routes map HTTP methods and paths to assignment controller operations.
ontroller layer	Implements list, retrieve, create, update, and delete operations and sends JSON responses.
Iodel layer	Mongoose schema describes fields, allowed status and priority values, and automatic timestamps.
atabase	MongoDB Atlas stores one document per assignment.
osting	GitHub Pages serves the static frontend; Render runs the backend service.

3.3 Data Model

The Assignment model stores a MongoDB document with an automatically generated identifier. The schema includes title, description, due date, status, priority, createdAt, and updatedAt. The title is required; description defaults to an empty string; status defaults to Pending; priority defaults to Medium; timestamps are enabled by Mongoose.

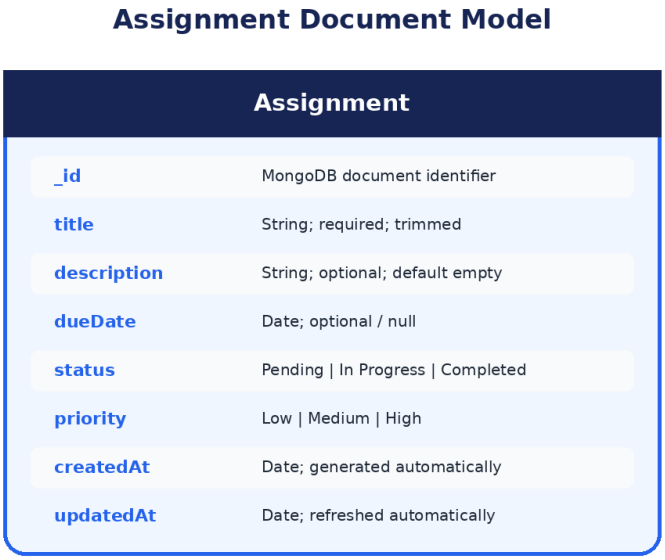


Figure 2. Assignment document fields and expected values.

3.4 Assignment Workflow

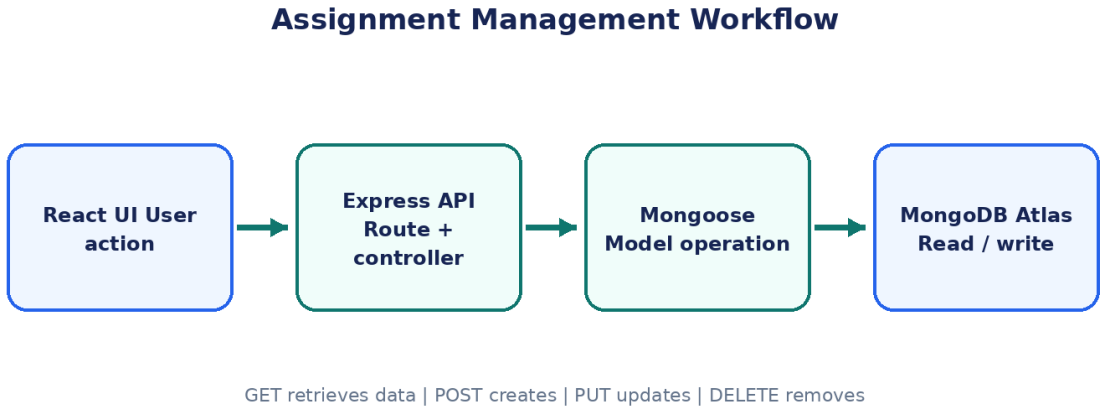


Figure 3. Typical request flow from the user interface to the database.

4. IMPLEMENTATION

4.1 Frontend Implementation

The frontend is a React single-page application built with Vite. React state is used to manage assignment data, loading state, error messages, search text, current status filter, modal visibility, selected record for editing, and form values. A useEffect hook loads the assignment list when the application mounts.

- Dashboard cards count total, pending, in-progress, and completed assignments.
- The assignment list displays title, description, due date, priority, and status.
- The search input matches title and description text.
- A status selector filters the visible list.
- A modal form supports creating and editing assignment data.
- Delete actions ask for confirmation before sending the DELETE request.
- API failures are shown in an error banner.

4.2 Backend Implementation

The backend is a Node.js application using Express. Middleware parses JSON request bodies and enables cross-origin requests through the cors package. A health route returns a JSON success message. Assignment routes are mounted at /api/assignments and delegate work to controller functions. The server connects to MongoDB through a dedicated database configuration module before listening on the port provided by the hosting environment.

4.3 API Endpoints

Method	Endpoint	Purpose
GET	/api/health	Health check; returns success message.
GET	/api/assignments	Returns all assignments, count, and data.
GET	/api/assignments/:id	Returns one assignment by identifier.
POST	/api/assignments	Creates a new assignment from JSON body.
PUT	/api/assignments/:id	Updates an existing assignment.
DELETE	/api/assignments/:id	Deletes an assignment.

4.4 Example Assignment Payload

```
{
  "title": "Data Structures Lab",
  "description": "Complete the linked-list exercises",
  "dueDate": "2026-11-05",
  "status": "Pending",
  "priority": "Medium"
}
```

Verification note: The deployed test confirmed assignment loading and create-and-refresh persistence. Edit, delete, search, and filter are implemented in the codebase and should be demonstrated on the live site before final submission.

5. TESTING AND VALIDATION

Testing was performed through browser checks of the deployed endpoints and the hosted web application. API endpoints were checked directly, and the frontend was tested for data loading and create-and-persist behavior. The table distinguishes outcomes confirmed during the project session from further checks recommended before final evaluation.

Test ID	Test	Expected result	Recorded status
TC-01	Open health endpoint	GET /api/health returns success JSON	Passed
TC-02	Load assignment list	GET /api/assignments returns count and data	Passed
TC-03	Load hosted frontend	GitHub Pages website opens	Passed
TC-04	Display assignments	Existing assignments appear in the frontend	Passed
TC-05	Create assignment	Submit form; API creates a record	Passed
TC-06	Persistence	Refresh page; created assignment remains	Passed
TC-07	Edit assignment	Update a record and confirm the change persists	Verify before submission
TC-08	Delete assignment	Confirm delete and verify record is removed	Verify before submission
TC-09	Search and filter	Search text and status filter change visible list	Verify before submission
TC-10	CORS / browser requests	Hosted frontend calls Render API successfully	Passed after deployment correction

5.1 Error Handling

The frontend reads the response body and surfaces a useful message when an HTTP response is not successful. A network-level failure such as Failed to fetch may occur when the server URL is wrong, the Render service is unavailable, or CORS preflight requests are not accepted. Browser developer tools and direct curl requests can be used to distinguish these cases.

5.2 Limitations of Testing

Testing was manual and focused on the main user journey. No automated unit-test suite, load test, penetration test, or cross-browser test matrix was run as part of the documented workflow. The edit, delete, search, and filter scenarios should be demonstrated during the final review.

6. DEPLOYMENT AND OPERATION

6.1 Deployment Topology

Layer	Platform	Deployment purpose
Frontend	GitHub Pages	Hosts the built static React/Vite assets.
Backend	Render Web Service	Runs Node.js/Express API and connects to MongoDB.
Database	MongoDB Atlas	Stores assignment records outside the frontend host.
Source control	GitHub	Stores source code and triggers frontend deployment workflow.

6.2 Local Setup

1. Install a current LTS version of Node.js and npm.
2. Clone the repository from the GitHub URL listed in this report.
3. Install frontend dependencies in client using npm install.
4. Install backend dependencies in server using npm install.
5. Create server/.env with the MongoDB connection string, named MONGODB_URI. Never commit this file.
6. Start the backend from server with npm start.
7. Start the frontend from client with npm run dev.
8. Open the local Vite URL shown in the terminal, normally http://localhost:5173.

6.3 Environment Configuration

Variable	Purpose	Required in Render
MONGODB_URI	MongoDB Atlas connection string	Yes
PORT	HTTP port; Render supplies this automatically	No manual value normally required

6.4 Deployment URLs

Hosted frontend: https://akshay225725-bit.github.io/student-assignment-tracker/

GitHub repository: https://github.com/akshay225725-bit/student-assignment-tracker

Backend health endpoint: https://student-assignment-tracker-api-ut5n.onrender.com/api/health

Assignments endpoint: https://student-assignment-tracker-api-ut5n.onrender.com/api/assignments

> **Credential safety**
>
> Do not place MongoDB credentials in frontend source code or GitHub. Configure MONGODB_URI in Render environment variables and keep server/.env excluded from version control.

7. CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

Student Assignment Tracker demonstrates a working full-stack approach to managing student tasks. The React interface organizes assignments and sends requests to an Express API. The API uses Mongoose to store assignment documents in MongoDB Atlas, while GitHub Pages and Render provide separate hosting for the frontend and backend. The project session confirmed that the deployed health check and list endpoints returned JSON, the hosted page displayed assignments, and newly created records persisted after a refresh.

7.2 Future Enhancements

- User registration and authentication, with ownership checks for assignment records.
- Due-date reminders through email, browser notifications, or a calendar integration.
- Sorting by deadline, title, priority, and most recently updated.
- File attachments, course/subject grouping, and assignment categories.
- Charts for weekly workload and completion trends.
- Automated API tests and continuous integration checks.
- More restrictive production CORS rules allowing only the known frontend origin.
- Pagination and database indexing for larger datasets.
- Accessibility checks and additional responsive/mobile testing.

REFERENCES

- **Project repository:** https://github.com/akshay225725-bit/student-assignment-tracker
- **Hosted application:** https://akshay225725-bit.github.io/student-assignment-tracker/
- **React documentation:** https://react.dev/
- **Vite documentation:** https://vite.dev/guide/
- **Node.js documentation:** https://nodejs.org/docs/latest/api/
- **Express documentation:** https://expressjs.com/
- **Mongoose documentation:** https://mongoosejs.com/docs/
- **MongoDB Atlas documentation:** https://www.mongodb.com/docs/atlas/
- **Render documentation:** https://render.com/docs
- **GitHub Pages documentation:** https://docs.github.com/pages